

# Back-end Frameworks

## Sessão do usuário

### Cadastro, Login e Recuperação de Senha

### Parte 2 – Segurança com JWT

Profº: Joseph Donald

Contatos:

 [www.josephdonald.wordpress.com](http://www.josephdonald.wordpress.com)

 @josephdonald

 030106382@prof.uninassau.edu.br

“O sucesso consiste em ir de fracasso em fracasso sem perder o entusiasmo.”  
Winston Churchill



## Como fazíamos antes?

### Antes:

O frontend enviava a chave estrangeira do usuário logado via JSON e a rota recebia a informação pela URL explicitamente.

Ex: *Envio do ID*

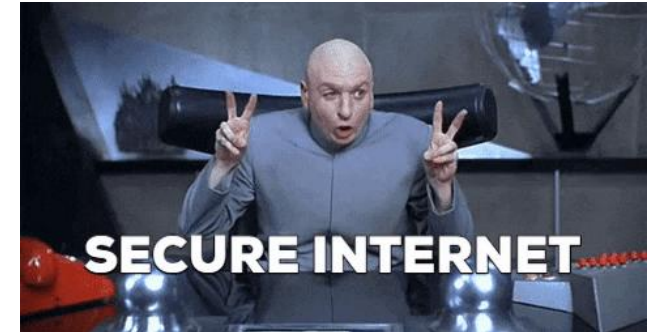
```
{  
  "idUserioFK": 1  
}
```

*Rota de leitura*

```
/lerFuncionario/1
```

### Problemas

- Insegurança
- Rota com ID exposto, onde poderia haver alteração do ID.
- Dependência do frontend enviar o ID correto





# Como faremos agora?

## Agora:

O token gerado pelo servidor Backend vai identificar o usuário. Ou seja, o usuário não será identificado por um ID, mas pelo seu token.

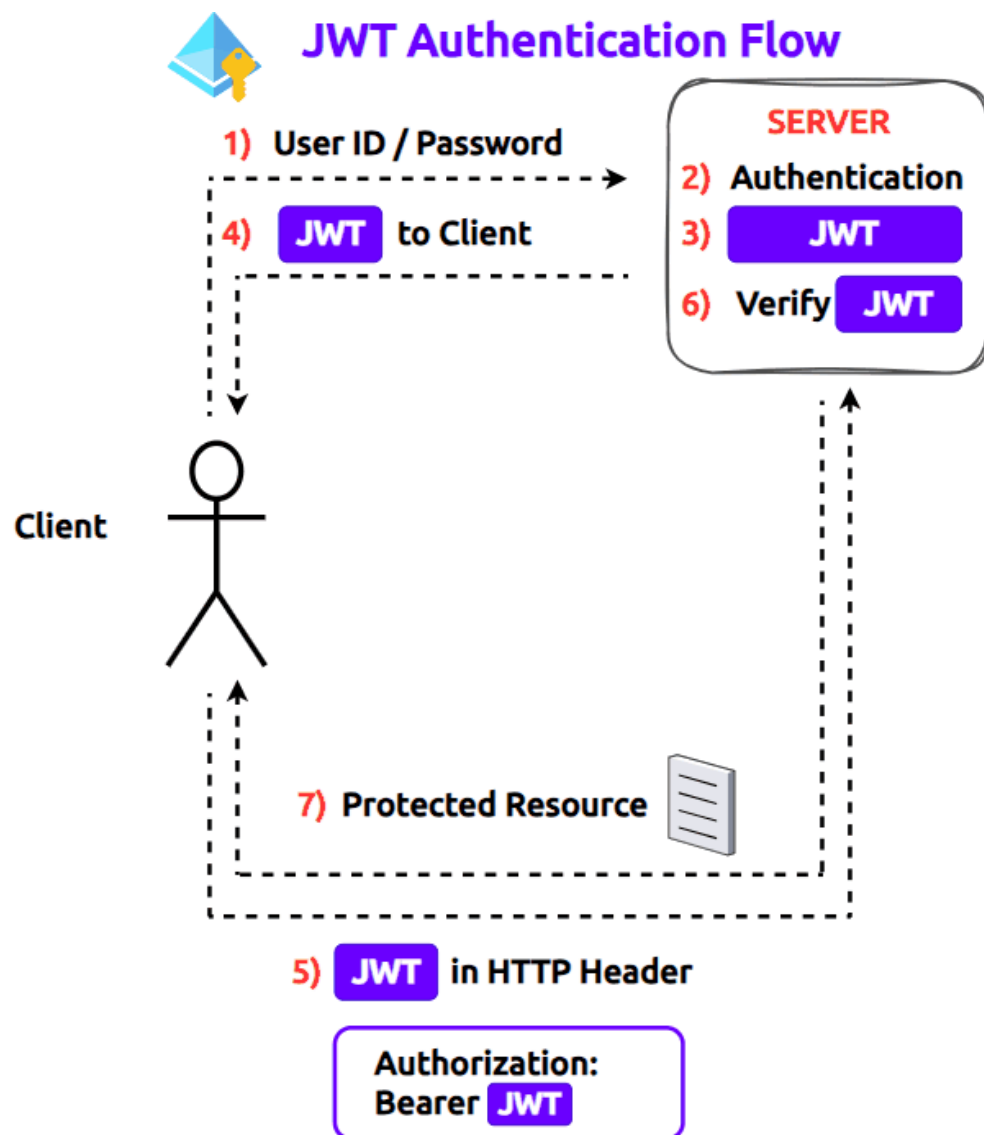
Assim, o Frontend envia no cabeçalho do pacote JSON a informação **Authorization: Bearer TOKEN**

```
var response = await http.post(  
  Uri.parse('http://$ip_servidor:5000/cadastraFuncionario'),  
  headers: {  
    'Accept': 'application/json',  
    'content-type': 'application/json',  
    'Authorization': 'Bearer $token', // Enviar o token JWT no  
  },  
  body: jsonEncode(mapFuncionario),  
  encoding: Encoding.getByName('utf-8'),  
);
```

Então o Flask vai descobrir quem é o usuário através do comando:

```
30 ## CREATE - CADASTRO DO FUNCIONARIO  
31 @app.route('/cadastraFuncionario', methods=['POST'])  
32 @jwt_required()  
33 def cadastraFuncionario():  
34     dadosRecebidos = request.get_json()  
35  
36     nome = dadosRecebidos["nomeFuncionario"]  
37     telefone = dadosRecebidos["telefoneFuncionario"]  
38     idUsuarioFK = get_jwt_identity()  
39     banco = conectaDB()  
40     cursor = banco.cursor()  
41  
42     sql = """INSERT INTO funcionario(nome, telefone, id_usuario_fk) VALUES (%s, %s, %s)"""
```







# Configurações necessárias

1º Instalar a lib **flask\_jwt\_extended** com o comando: `>pip install flask-jwt-extended`

2º Importar para dentro do projeto a lib através da linha:

```
2 # importar a JWT para criar tokens de autenticação
3 from flask_jwt_extended import JWTManager, create_access_token, jwt_required, get_jwt_identity
```

3º Configure na instância do **app** a chave secreta:

- 'JWT\_SECRET\_KEY' com uma palavra chave a seu critério
- Configure o tempo de expiração do token em 'JWT\_ACCESS\_TOKEN\_EXPIRES'
- Instancie o **app** na classe **JWTManager( )**

```
13 app = Flask(__name__)
14 app.config['JWT_SECRET_KEY'] = 'abracadabra' # Substitua por uma chave secreta forte
15 app.config['JWT_ACCESS_TOKEN_EXPIRES'] = timedelta(days=1) # Define o tempo de expiração do token para 1 minuto
16 jwt = JWTManager(app) # Inicializa a extensão JWT com a aplicação Flask
17 CORS(app)
```

4º Instale a lib Flask-Cors, ela permite que sua API receba requisições HTTP de domínios (sites ou aplicativos frontend) diferentes daquele onde a API está hospedada. Para isso utilize o comando `pip install Flask-Cors`

5º Insira a linha **CORS(app)** logo após a linha do JWTManager( )



## Criação do TOKEN no momento do Login

```
165 ## LOGIN - ROTA PARA REALIZAR O LOGIN DO USUÁRIO recolhendo todas as informações do usuário para realizar a autenticação
166 @app.route('/loginUsuario', methods=['POST'])
167 def loginUsuario():
168     dadosRecebidos = request.get_json()
169     email = dadosRecebidos["emailUsuario"]
170     senha = dadosRecebidos["senhaUsuario"]
171
172     banco = conectaDB()
173     cursor = banco.cursor()
174
175     #implementar criptografia da senha utilizando a biblioteca bcrypt, comparando a senha recebida com a senha
176     # armazenada no banco de dados utilizando a função bcrypt.checkpw
177     sql = """SELECT id_usuario, nome, email, senha FROM usuario WHERE email = %s"""
178     try:
179         cursor.execute(sql, (email,))
180         resultado = cursor.fetchone()
181         if resultado:
182             # verificar a senha utilizando o bcrypt
183             if bcrypt.checkpw(senha.encode('utf-8'), resultado[3].encode('utf-8')):
184                 # criar o token de autenticação utilizando a função create_access_token da biblioteca Flask-JWT-Extended,
185                 # passando o id do usuário como identidade do token
186                 token = create_access_token(identity=str(resultado[0]))
187                 # definir o tempo de expiração do token para 1 hora
188
189                 response = {"mensagem" : "Login realizado com sucesso", "usuario" : {
190                     "id" : resultado[0],
191                     "nome" : resultado[1],
192                     "email" : resultado[2]
193                 }, "token" : token}
194                 return jsonify(response), 200
```

Na linha 186, após a confirmação de login através do e-mail e senha, o Flask aciona a função **create\_access\_token()** recebendo como parâmetro o ID do usuário que foi obtido durante a consulta de e-mail e senha.

O token é gerado e devolvido para o usuário através de JSON, conforme a linha 193.



Para cada rota que o programador quiser proteger com JWT, deverá ser inserida a assinatura `@jwt_required( )` e o token vai ser obtido através da função `get_jwt_identity( )`

Ex:

```
30  ## CREATE - CADASTRO DO FUNCIONARIO
31  @app.route('/cadastraFuncionario', methods=['POST'])
32  @jwt_required()
33  def cadastraFuncionario():
34      dadosRecebidos = request.get_json()
35
36      nome = dadosRecebidos["nomeFuncionario"]
37      telefone = dadosRecebidos["telefoneFuncionario"]
38      idUsuarioFK = get_jwt_identity()
39      banco = conectaDB()
40      cursor = banco.cursor()
41
```





# Criptografando a senha do usuário

- Para uma boa segurança, devemos criptografar dados sensíveis.  
Ex: senha que será salva no banco.
- Para instalar o bcrypt, utilize com comando: `pip install bcrypt`
- Para criptografar este tipo de dados utilizamos o **bcrypt**. Assim, importe o bcrypt no arquivo em que será utilizado com o comando:

```
10 # importar a biblioteca bcrypt para realizar o hash das senhas dos usuários
11 import bcrypt
```

- O bcrypt transforma a senha em HASH através do comando:

```
152 # realizar o hash da senha antes de armazená-la
153 senha_hash = bcrypt.hashpw(senha.encode('utf-8'), bcrypt.gensalt())
```

- Para fazer a verificação de senha no login, o comando no bcrypt Codifica a senha vinda do frontend e compara com a senha existente no banco de dados.

```
181 # verificar a senha utilizando o bcrypt
182 if bcrypt.checkpw(senha.encode('utf-8'), resultado[3].encode('utf-8')):
```





# Realize as seguintes alterações no código

```
main.py > loginUsuario
1 from urllib import response
2 # importar a JWT para criar tokens de autenticação
3 from flask_jwt_extended import JWTManager, create_access_token, jwt_required, get_jwt_identity
4 from flask import Flask, request, jsonify
5 import pymysql
6 from flask_mail import Mail, Message
7 import random
8 from flask_cors import CORS
9 from datetime import timedelta
10 # importar a biblioteca bcrypt para realizar o hash das senhas dos usuários
11 import bcrypt
12
13 app = Flask(__name__)
14 app.config['JWT_SECRET_KEY'] = 'abracadabra' # Substitua por uma chave secreta forte
15 app.config['JWT_ACCESS_TOKEN_EXPIRES'] = timedelta(days=1) # Define o tempo de expiração do token para 1 minuto
16 jwt = JWTManager(app) # Inicializa a extensão JWT com a aplicação Flask
17 CORS(app)
18
19 def conectaDB():
20     db = pymysql.connect(
21         host='localhost',
22         database='banco_funcionarios',
23         user='root',
24         passwd=''
25     )
26
27     return db
28
29
30 ## CREATE - CADASTRO DO FUNCIONARIO
31 @app.route('/cadastraFuncionario', methods=['POST'])
32 @jwt_required()
33 def cadastraFuncionario():
34     dadosRecebidos = request.get_json()
35
36     nome = dadosRecebidos["nomeFuncionario"]
37     telefone = dadosRecebidos["telefoneFuncionario"]
38     idUsuarioFK = get_jwt_identity()
39     banco = conectaDB()
40     cursor = banco.cursor()
41
42     sql = """INSERT INTO funcionario(nome, telefone, id_usuario_fk) VALUES (%s, %s, %s)"""
43     try:
44         cursor.execute(sql, (nome, telefone, idUsuarioFK))
45         banco.commit()
46         response = {"mensagem": "Cadastrado com sucesso"}, 200
47         return jsonify(response)
48     except Exception as e:
49         response = {"mensagem": "Erro ao cadastrar funcionario", "erro": str(e)}, 400
50         return jsonify(response)
51     finally:
52         banco.close()
53
```

```
main.py > cadastraFuncionario
54
55 # rota para atualização dos dados do funcionário nomeada como /atualizaFuncionario, utilizando o método PUT
56 @app.route('/atualizaFuncionario', methods=['PUT'])
57 @jwt_required()
58 def atualizaFuncionario():
59     dadosRecebidos = request.get_json()
60     idFuncionario = dadosRecebidos["idFuncionario"]
61     nomeFuncionario = dadosRecebidos["nomeFuncionario"]
62     telefone = dadosRecebidos["telefoneFuncionario"]
63     idUsuarioFK = get_jwt_identity()
64
65     banco = conectaDB()
66     cursor = banco.cursor()
67
68     sql = """UPDATE funcionario SET nome = %s, telefone = %s WHERE id_funcionario = %s AND id_usuario_fk = %s"""
69     try:
70         cursor.execute(sql, (nomeFuncionario, telefone, idFuncionario, idUsuarioFK))
71         banco.commit()
72         response = {"mensagem": "Atualizado com sucesso"}, 200
73         return jsonify(response)
74     except Exception as e:
75         response = {"mensagem": "Erro ao atualizar funcionário", "erro": str(e)}, 400
76         return jsonify(response)
77     finally:
78         banco.close()
79
80
81 ## READ - LER OS FUNCIONARIOS DO BANCO somente do usuário logado, utilizando o método GET
82 # e recebendo o id do usuário como parâmetro na URL
83 @app.route('/lerFuncionario', methods=['GET'])
84 @jwt_required()
85 def lerFuncionario():
86     id_usuario = get_jwt_identity()
87     banco = conectaDB()
88     cursor = banco.cursor()
89
90     sql = """SELECT * FROM funcionario WHERE id_usuario_fk = %s;"""
91     try:
92         cursor.execute(sql, (id_usuario,))
93         resultado = cursor.fetchall()
94         response = {"mensagem": "Funcionários lidos com sucesso"}, 200
95     except Exception as e:
96         response = {"mensagem": "Erro ao ler funcionarios", "erro": str(e)}, 400
97     finally:
98         banco.close()
99
100     listaFuncionario = []
101
102     for funcionario in resultado:
103         listaFuncionario.append(
104             {
105                 "id": funcionario[0],
106                 "nomeFuncionario": funcionario[1],
107                 "telefoneFuncionario": funcionario[2],
108                 "idUsuarioFK": funcionario[3]
109             }
110         )
111
112     return jsonify(listaFuncionario, response)
113
```



```
113
114 # Rota para deletar um funcionário utilizando o método DELETE
115 @app.route('/deletaFuncionario', methods=['DELETE'])
116 @jwt_required()
117 def deletaFuncionario():
118     dadosRecebidos = request.get_json()
119     idFuncionario = dadosRecebidos["idFuncionario"]
120     idUsuarioFK = get_jwt_identity()
121
122     banco = conectaDB()
123     cursor = banco.cursor()
124
125     sql = """DELETE FROM funcionario WHERE id_funcionario = %s AND id_usuario_fk = %s"""
126     try:
127         cursor.execute(sql, (idFuncionario, idUsuarioFK))
128         banco.commit()
129         response = {"mensagem": "Deletado com sucesso"}, 200
130     except Exception as e:
131         response = {"mensagem": "Erro ao deletar funcionário", "erro": str(e)}, 400
132     finally:
133         banco.close()
134
135     return jsonify(response)
136
137 ##### CRUD DE USUÁRIO E AUTENTICAÇÃO #####
138
139 ## CREATE - ROTA PARA CADASTRAR UM USUÁRIO
140 @app.route('/cadastraUsuario', methods=['POST'])
141 def cadastraUsuario():
142     dadosRecebidos = request.get_json()
143     nome = dadosRecebidos["nomeUsuario"]
144     email = dadosRecebidos["emailUsuario"]
145     senha = dadosRecebidos["senhaUsuario"]
146
147     banco = conectaDB()
148     cursor = banco.cursor()
149
150     sql = """INSERT INTO usuario(nome, email, senha) VALUES (%s, %s, %s)"""
151     try:
152         # realizar o hash da senha antes de armazená-la
153         senha_hash = bcrypt.hashpw(senha.encode('utf-8'), bcrypt.gensalt())
154         cursor.execute(sql, (nome, email, senha_hash))
155         banco.commit()
156         response = {"mensagem": "Cadastrado com sucesso"}, 200
157         return jsonify(response)
158     except pymysql.err.IntegrityError as e:
159         response = {"mensagem": "Email já cadastrado", "erro": str(e)}, 400
160         return jsonify(response)
161     finally:
162         banco.close()
163
164
```



```
164
165 ## LOGIN - ROTA PARA REALIZAR O LOGIN DO USUÁRIO recolhendo todas as informações do usuário para realizar a autenticação
166 @app.route('/loginUsuario', methods=['POST'])
167 def loginUsuario():
168     dadosRecebidos = request.get_json()
169     email = dadosRecebidos["emailUsuario"]
170     senha = dadosRecebidos["senhaUsuario"]
171
172     banco = conectaDB()
173     cursor = banco.cursor()
174
175     #implementar criptografia da senha utilizando a biblioteca bcrypt, comparando a senha recebida com a senha armazenada no banco de dados utilizando a função bcrypt.checkpw
176     sql = """SELECT id_usuario, nome, email, senha FROM usuario WHERE email = %s"""
177     try:
178         cursor.execute(sql, (email,))
179         resultado = cursor.fetchone()
180         if resultado:
181             # verificar a senha utilizando o bcrypt
182             if bcrypt.checkpw(senha.encode('utf-8'), resultado[3].encode('utf-8')):
183                 # criar o token de autenticação utilizando a função create_access_token da biblioteca Flask-JWT-Extended, passando o id do usuário como identidade do token
184                 token = create_access_token(identity=str(resultado[0]))
185                 # definir o tempo de expiração do token para 1 hora
186
187                 response = {"mensagem" : "Login realizado com sucesso", "usuario" : {
188                     "id" : resultado[0],
189                     "nome" : resultado[1],
190                     "email" : resultado[2]
191                 }, "token" : token}
192                 return jsonify(response), 200
193             else:
194                 response = {"mensagem" : "E-mail ou senha incorretos"}
195                 return jsonify(response), 401
196         except Exception as e:
197             response = {"mensagem" : "Ocorreu um erro ao tentar fazer login", "erro" : str(e)}
198             return jsonify(response), 401
199         finally:
200             banco.close()
201
```



```
201
202  ## rota de recuperação de senha através do Flask-Mail, utilizando o método POST para enviar um e-mail de recuperação com um código temporário de confirmação
203  @app.route('/recuperaSenha', methods=['POST'])
204  def recuperaSenha():
205
206      global codigo_recuperacao_gerado
207
208      dadosRecebidos = request.get_json()
209      email = dadosRecebidos["emailUsuario"]
210
211      banco = conectaDB()
212      cursor = banco.cursor()
213
214      sql = """SELECT id_usuario, nome, email FROM usuario WHERE email = %s"""
215      try:
216          cursor.execute(sql, (email,))
217          resultado = cursor.fetchone()
218          if resultado:
219
220              app.config['MAIL_SERVER'] = 'smtp.gmail.com'
221              app.config['MAIL_PORT'] = 587
222              app.config['MAIL_USE_TLS'] = True
223              app.config['MAIL_USERNAME'] = 'josephdonalddti@gmail.com'
224              app.config['MAIL_PASSWORD'] = 'coyz fwjt wqbl kvtv'
225              mail = Mail(app)
226
227              msg = Message("Recuperação de Senha", sender="josephdonalddti@gmail.com", recipients=[email])
228              codigo_recuperacao_gerado = random.randint(100000, 999999)
229              msg.body = f"Olá seu código de recuperação de senha é: {codigo_recuperacao_gerado}"
230
231              mail.send(msg)
232
233              return jsonify(response = {"mensagem" : "E-mail de recuperação enviado com sucesso", "usuario" : {
234                  "id" : resultado[0],
235                  "nome" : resultado[1],
236                  "email" : resultado[2]
237              }}, 200)
238          else:
239              response = {"mensagem" : "E-mail não encontrado"}, 401
240              return jsonify(response)
241
242      except Exception as e:
243          response = {"mensagem" : "Erro ao recuperar senha", "erro" : str(e)}, 400
244          return jsonify(response)
245      finally:
246          banco.close()
247
```



```
247
248 # Rota para atualizar a senha do usuário utilizando o método PUT, recebendo o email do usuário e a nova senha
249 @app.route('/atualizaSenha', methods=['PUT'])
250 def atualizaSenha():
251     dadosRecebidos = request.get_json()
252     email = dadosRecebidos["emailUsuario"]
253     nova_senha = dadosRecebidos["novaSenha"]
254     codigo_recuperacao_recebido = dadosRecebidos["codigoRecuperacao"]
255
256     print("Email recebido para atualização de senha:", email)
257     print("Nova senha recebida:", nova_senha)
258     print("Código de recuperação recebido:", codigo_recuperacao_recebido)
259     print("Código de recuperação gerado:", codigo_recuperacao_gerado)
260
261     if codigo_recuperacao_recebido != codigo_recuperacao_gerado:
262         return jsonify({"mensagem": "Código de recuperação inválido"}), 401
263
264     banco = conectaDB()
265     cursor = banco.cursor()
266     #implementar a atualização da senha utilizando a biblioteca bcrypt para realizar o hash da nova senha antes de armazená-la no banco de dados
267     nova_senha_hash = bcrypt.hashpw(nova_senha.encode('utf-8'), bcrypt.gensalt())
268     sql = """UPDATE usuario SET senha = %s WHERE email = %s"""
269     try:
270         cursor.execute(sql, (nova_senha_hash, email))
271         banco.commit()
272         response = {"mensagem": "Senha atualizada com sucesso."}, 200
273         return jsonify(response)
274
275     except Exception as e:
276         response = {"mensagem": "Erro ao atualizar a senha"}, 400
277         return jsonify(response)
278
279     finally:
280         banco.close()
281
282 if __name__ == "__main__":
283     app.run(debug=True, host='0.0.0.0')
284
285
```